

DETAILED THEORETICAL REPORT

MODULE 2 – UNIT TEST FRAMEWORKS

Name: Saheli Deb

Enrollment No.: 12023052019071

Course: Python Automation

Report Type: Section-wise Theoretical Report with Illustrative Code

2.1 unittest Framework

Python's built-in unittest module provides a structured framework for writing and executing automated tests. It follows the xUnit family of testing frameworks and organizes related test cases inside classes derived from `unittest.TestCase`.

A test method normally starts with `test_`, allowing the unittest test loader to discover it automatically. Assertions are implemented through methods such as `assertEqual()`, `assertNotEqual()`, `assertTrue()`, `assertFalse()`, `assertIn()` and `assertRaises()`.

The test lifecycle can be controlled through `setUp()` and `tearDown()`. `setUp()` executes before each test method and is useful for preparing test data or creating objects. `tearDown()` executes after each test and is used for cleanup. For class-level setup, unittest also provides `setUpClass()` and `tearDownClass()`.

unittest supports test suites, test runners and command-line execution. A suite groups multiple test cases, while the runner executes the selected tests and reports pass/fail status. This makes unittest appropriate for both small unit tests and structured automation projects.

Basic unittest test case

```
import unittest

class Calculator:
    def add(self, a, b):
        return a + b

class TestCalculator(unittest.TestCase):

    def setUp(self):
        self.calculator = Calculator()

    def test_add(self):
        result = self.calculator.add(10, 20)
        self.assertEqual(result, 30)

    def tearDown(self):
        self.calculator = None

if __name__ == "__main__":
    unittest.main()
```

Common assertions

```
import unittest

class TestAssertions(unittest.TestCase):

    def test_assertions(self):
        self.assertEqual(5 + 5, 10)
        self.assertNotEqual(5, 6)
        self.assertTrue(10 > 5)
        self.assertFalse(2 > 8)
        self.assertIn("Python", "Python Automation")

    def test_exception(self):
        with self.assertRaises(ZeroDivisionError):
            10 / 0
```

2.2 PyTest Framework

PyTest is a widely used Python testing framework designed to make test creation concise and expressive. Unlike unittest, PyTest allows ordinary Python functions to be used as tests, and the built-in `assert` statement is used for validation.

PyTest discovers tests according to naming conventions. Test files commonly begin with `test_` or end with `_test.py`, while test functions and methods generally begin with `test_`. The framework provides detailed assertion output, fixtures, parametrization, markers, skipping, expected failures and a large plugin ecosystem.

One of the major advantages of PyTest is that setup and teardown can be expressed through fixtures. Fixtures can have different scopes such as function, class, module or session. This enables resources such as WebDriver instances, database connections or API sessions to be shared efficiently.

PyTest also supports command-line execution. The `-v` option provides verbose output, `-k` selects tests using an expression, `-m` selects tests by marker, and `--maxfail` can stop execution after a specified number of failures.

Basic PyTest tests

```
def add(a, b):
    return a + b

def test_add():
    assert add(2, 3) == 5

def test_add_negative():
    assert add(-2, -3) == -5

def test_add_zero():
    assert add(10, 0) == 10

# Run:
# pytest -v
```

PyTest exception assertion

```
import pytest

def divide(a, b):
    return a / b

def test_divide_by_zero():
    with pytest.raises(ZeroDivisionError):
        divide(10, 0)
```

Useful PyTest commands

```
# Run all tests
pytest

# Verbose execution
pytest -v

# Run a specific file
pytest -v test_login.py

# Run tests matching a keyword
pytest -v -k login

# Stop after first failure
pytest -x
```

2.3 Fixtures and conftest.py

Fixtures are reusable pieces of setup and teardown logic. They are one of PyTest's most important features because they prevent repeated initialization code across test functions.

A fixture is created with `@pytest.fixture`. When a test function includes the fixture name as an argument, PyTest automatically supplies the fixture value. If the fixture contains a `yield`, statements before `yield` represent setup and statements after `yield` represent teardown.

Fixture scope controls how frequently the fixture is created. Function scope creates one instance per test function. Class scope creates one per test class, module scope creates one per module, and session scope creates one for the complete test session.

The `conftest.py` file is a special PyTest configuration file used to store fixtures and hooks that should be automatically available to tests in the same directory hierarchy. It avoids importing fixtures manually into every test module.

Basic fixture

```
import pytest

@pytest.fixture
def user():
    return {
        "name": "Saheli",
        "role": "Tester"
    }

def test_user_name(user):
    assert user["name"] == "Saheli"

def test_user_role(user):
    assert user["role"] == "Tester"
```

Fixture with teardown using yield

```
import pytest

@pytest.fixture
def database():
    print("Connect to database")
    db = {"status": "connected"}

    yield db

    print("Close database")
    db["status"] = "closed"

def test_database(database):
    assert database["status"] == "connected"
```

conftest.py pattern

```
# conftest.py
import pytest

@pytest.fixture
def browser_name():
    return "Chrome"

# test_browser.py
def test_browser(browser_name):
    assert browser_name == "Chrome"

# No import of browser_name is required.
```

2.4 Parameterization and Data-Driven Testing

Parameterization allows the same test logic to execute against multiple input combinations. Instead of creating separate test functions for every data set, PyTest can generate multiple test cases from a single function.

The `@pytest.mark.parametrize` decorator accepts argument names and a list of values. Each value set becomes an independent test invocation. This is useful for valid and invalid login combinations, boundary values, API payloads, browser configurations and other test data.

Data-driven testing improves coverage while reducing duplicated code. The expected result should be included with the input when the test needs different outcomes for different cases.

Basic parameterization

```
import pytest

@pytest.mark.parametrize(
    "a,b,expected",
    [
        (2, 3, 5),
        (10, 20, 30),
        (-2, 5, 3),
        (0, 7, 7),
    ]
)
def test_add(a, b, expected):
    assert a + b == expected
```

Login data-driven test

```
import pytest

@pytest.mark.parametrize(
    "username,password,expected",
    [
        ("standard_user", "secret_sauce", True),
        ("wrong_user", "wrong_pass", False),
        ("standard_user", "wrong_pass", False),
    ]
)
def test_login(username, password, expected):
    actual = (
        username == "standard_user"
        and password == "secret_sauce"
    )
    assert actual is expected
```

2.5 PyTest with Selenium and HTML Reporting

PyTest can be combined with Selenium to build browser automation suites. A browser driver is commonly provided through a fixture so that each test receives a fresh browser and the browser is closed after execution.

This combination separates test lifecycle management from individual test cases. The fixture owns browser setup and cleanup, while the test focuses on the business behavior being validated.

PyTest can generate HTML reports through the `pytest-html` plugin. Reports provide a readable summary of test results and can be stored as build artifacts in CI/CD pipelines. Screenshots can also be captured when a UI test fails.

Selenium fixture

```
import pytest
from selenium import webdriver

@pytest.fixture
def driver():
    browser = webdriver.Chrome()
    yield browser
    browser.quit()

def test_saucedemo_title(driver):
    driver.get("https://www.saucedemo.com/")
    assert "Swag Labs" in driver.title
```

HTML report

```
# Install
# pip install pytest pytest-html selenium

# Run
# pytest -v --html=report.html --self-contained-html
```

2.6 Page Object Model (POM)

The Page Object Model is a design pattern for UI automation. Each important page or reusable component is represented by a Python class. The class contains locators and methods that describe how to interact with that page.

The test itself should ideally express business-level behavior rather than low-level locator details. For example, a test should say `login\_page.login(username, password)` instead of repeating three `find\_element()` calls.

POM reduces maintenance cost. If a locator changes, the corresponding page class can be updated without modifying every test that uses the page. It also encourages reusable components such as LoginPage, DashboardPage, ProductPage and CheckoutPage.

A BasePage can hold common browser operations such as click, type, wait, screenshot and navigation. Specialized page classes inherit or use those utilities.

Login Page Object

```
from selenium.webdriver.common.by import By

class LoginPage:
    USERNAME = (By.ID, "user-name")
    PASSWORD = (By.ID, "password")
    LOGIN_BUTTON = (By.ID, "login-button")

    def __init__(self, driver):
        self.driver = driver

    def enter_username(self, username):
        self.driver.find_element(*self.USERNAME).send_keys(username)

    def enter_password(self, password):
        self.driver.find_element(*self.PASSWORD).send_keys(password)

    def click_login(self):
        self.driver.find_element(*self.LOGIN_BUTTON).click()

    def login(self, username, password):
        self.enter_username(username)
        self.enter_password(password)
        self.click_login()
```

Test using POM

```
def test_valid_login(driver):
    from login_page import LoginPage

    driver.get("https://www.saucedemo.com/")

    login_page = LoginPage(driver)
    login_page.login("standard_user", "secret_sauce")

    assert "/inventory.html" in driver.current_url
```

BasePage concept

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class BasePage:
    def __init__(self, driver):
        self.driver = driver
        self.wait = WebDriverWait(driver, 10)

    def click(self, locator):
        self.wait.until(
            EC.element_to_be_clickable(locator)
        ).click()

    def type_text(self, locator, text):
        element = self.wait.until(
            EC.visibility_of_element_located(locator)
        )
        element.clear()
        element.send_keys(text)
```

2.7 Test Organization, Markers, Skipping and Reporting

As a test suite grows, tests should be organized by functionality, layer and execution purpose. PyTest markers provide metadata that can be used to select groups such as smoke, regression or API tests.

Tests can be skipped when a feature is not available in a particular environment. Expected failures can be marked when a known issue is being tracked. Custom markers should be registered in the project's PyTest configuration to avoid warnings.

Reporting can include standard terminal output, HTML reports and plugins for XML or Allure-compatible results. In continuous integration, reports and screenshots are usually preserved as build artifacts.

Markers and skip

```
import pytest

@pytest.mark.smoke
def test_home_page():
    assert True

@pytest.mark.regression
def test_checkout():
    assert True

@pytest.mark.skip(reason="Feature not available")
def test_unavailable_feature():
    assert False

@pytest.mark.xfail(reason="Known defect")
def test_known_failure():
    assert False
```

Selecting tests

```
# Run smoke tests
# pytest -m smoke -v

# Run regression tests
# pytest -m regression -v

# Run everything except smoke
# pytest -m "not smoke" -v
```

Module 2 – Summary

This module explains the major Python testing frameworks and design practices used in automation. unittest provides a standard-library xUnit approach, while PyTest provides concise assertions, fixtures, parameterization and a rich plugin ecosystem. The Page Object Model introduces a maintainable architecture for Selenium tests by separating UI implementation from business-level test scenarios.

Key Learning Outcomes

- Create structured tests using Python unittest.
- Write concise PyTest test functions and assertions.
- Design reusable fixtures and manage setup/teardown using conftest.py.
- Apply parameterization for data-driven testing.
- Integrate Selenium WebDriver with PyTest.
- Generate HTML reports and preserve failure evidence.
- Implement Page Object Model classes and reusable BasePage utilities.
- Organize tests using markers, skipping and expected-failure mechanisms.

Conclusion: A maintainable automation framework is not only a collection of test scripts. It requires a clear testing structure, reusable setup, meaningful assertions, externalized test data, clean page abstractions and useful reporting. unittest and PyTest provide the execution foundation, while fixtures and POM provide the engineering structure needed for larger automation suites.